

第十节 · 从需求分析到任务拆解：拿到项目如何从零思考

拿到项目， 先想清楚怎么做成。

交付的不是代码，是一条从零思考的路径。

传统编程：做法和思路

想清楚需求 → 设计 → 一行行写代码 → 调试，核心动作是「写代码」。

01

理需求

搞清楚要做什么



02

设计

定架构与方案



03

写代码

一行行手撸实现



04

调试

一个个 bug 修

思考始终绕着「这段代码怎么实现」转。

衡量一个工程师的，是代码量、提交数、解决的 bug 数——写得多、写得快、写得准，就是好工程师。

AI 编程：做法和思路

核心动作变成「驾驭 AI 把事做成」——执行交给 AI。

01

想清楚

要做什么、怎么拆



02

立规范

给 AI 边界和规则



03

AI 执行

在边界内生成代码



04

你验收

判断对不对、收口

思考绕着「怎么更快更好地把事做成」转，不是「怎么写代码」。

代码还是产出物，但它从你手里直接流出，变成「你判断 → AI 执行 → 你验收」这条新链路产生。

哪些变了，哪些没变

看清这个，你既不会停在旧认知里，也不会被「AI 取代程序员」带偏。

没变

本质一直如此

- 目的：更快更好地产生价值
- 动手前要先想清楚需求
- 工程判断力依然是核心
- 计算机基础照样躲不掉

变了

路径与重心

- 核心动作：写代码 → 驾驭 AI
- 思考重心：实现细节 → 设计·拆解·验证
- 衡量标准：代码量 → 把事做成
- 靠的从「手感」变成「判断」

目的没变，变的是路径——这一节，就讲清楚新路径上「该怎么想」。

换个角度：用三种思维，不是写代码思维

拿到项目，别从「第一行代码」想起，从「怎么做成」想起。

不再问「这段代码怎么写」，而是问「这件事怎么更快更好地做成」。

从「做出东西」出发——用 SDD、Harness、Loop 三种思维去想，而不是用写代码的思维。

SDD

怎么拆分

Harness

怎么立规范

Loop

怎么自动化

这三种思维（第五节的三大支柱），现在变成你「拿到项目怎么想」的三个角度。

三种思维：拿到项目这样想

同一个项目，从三个角度想一遍，比从「第一行代码」想清楚得多。

SDD 思维

怎么拆分

先想「拆成什么、规格是什么」——把模糊需求拆成清晰、可执行的块。

Harness 思维

怎么立规范

先想「给 AI 什么边界、规则、怎么验证」——把约束架好，AI 才不跑偏。

Loop 思维

怎么自动化

想「哪些步骤让 AI 在循环里自己跑、哪里设检查点」——把执行循环设计好。

注意：这是「想」的三个角度，不是三个工具操作——先在脑子里想清楚，再落到工具。

一条思考路径：先做什么、后做什么

顺序不是死的，但这套骨架最稳。



先想清楚、再立约束、再让 AI 跑、最后人收口——执行那一步交给 AI，其余三步是你的。

但控制权，在你手里

三种思维是为了让 AI 高效执行，不是让 AI 替你做决定。

你定的

判断不能交出去

- 拆分方案：拆成什么块
- 规范与边界：什么能做、什么不能
- 关键技术决策：选哪条路线
- 验收标准：什么算做对了

AI 做的

执行可以交出去

- 在你划的边界内生成代码
- 按规格逐个执行任务
- 做完自动跑检查
- 卡住时把情况报告给你

你是工作流的设计者，不是旁观者——执行可以交出去，判断不能。

思考方式变了， 实现方式也变了。

拿到项目，先用 SDD / Harness / Loop 的思维想清楚，再让 AI 去做。

下一节 · 演示课

第十节：部署 OryxOS 官网，先给项目一个门面

- 这节是演示课，不需要 PPT
- 动手开发 & 部署 & 运行一个项目主页
- 把这节的思考路径，第一次落到真实动手上